Scenario 5 — API Testing

Prompt without Framework

Weak Prompt: Write API test cases for user registration

Response Received:

Here's a comprehensive set of API test cases for a **User Registration** endpoint (POST /api/users/register), covering functional, negative, edge, and security scenarios.

Created a file, read a file

41 test cases across functional, negative, boundary, security, and non-functional categories, with test IDs and priorities so you can drop them straight into ADO Test Plans. Let me know the actual field names/validation rules from your API spec if you want these tightened up to match exactly.

Prompt with Framework:

Act as integration test engineer and create positive test cases to validate the Post API endpoint for user registration functionality and generate the output in table view for quick reference. make sure you design the test case in a simple language. Use the username as - yuviofficial4491@gmail.com to create test script

Response Received:

Architected comprehensive test cases and validation script for registration endpoint
Architected comprehensive test cases and validation script for registration endpoint
Here are positive test cases for the **POST /api/users/register** endpoint, written in simple language using your test email throughout.

Test Data used: yuviofficial4491@gmail.com

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result
TC_POS_01	Register a new user with all required details	Send a POST request to /register with first name, last name, email, password, and confirm password	firstName: Yuvarani, lastName: R, email: yuviofficial4491@gmail.com, password: Test@1234, confirmPassword: Test@1234	API returns status code 201 (Created) with a success message and a new user ID
TC_POS_02	Register a new user with an optional phone number	Send a POST request with all required fields plus a valid phone number	email: yuviofficial4491@gmail.com, phone: 9876543210	API returns 201 Created; phone number is saved correctly
TC_POS_03	Check that the password is not shown in the response	Register using valid details, then check the response body	email: yuviofficial4491@gmail.com, password: Test@1234	Response does not contain the password anywhere in plain text

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result
TC_POS_04	Check that the email is saved correctly	Register a new user, then fetch the user details using the returned user ID	email: yuviofficial4491@gmail.com	Saved email matches yuviofficial4491@gmail.com exactly
TC_POS_05	Register with the shortest allowed password	Send a POST request using a password at the minimum allowed length	email: yuviofficial4491@gmail.com, password: Test@12 (minimum length)	API returns 201 Created; registration is successful
TC_POS_06	Verify a welcome/verification email is sent	Register a new user and check the mailbox for the given email	email: yuviofficial4491@gmail.com	A verification or welcome email is received at yuviofficial4491@gmail.com
TC_POS_07	Verify the new user can log in right after registering	Register the user, then call the login API with the same credentials	email: yuviofficial4491@gmail.com, password: Test@1234	Login is successful and returns a valid access token
TC_POS_08	Verify the correct response format is returned	Register a new user and check the response headers and body	email: yuviofficial4491@gmail.com	Content-Type is application/json; response includes userId, email, and createdAt
TC_POS_09	Verify the new user ID can be used to fetch the profile	Register a new user, then send a GET request using the returned user ID	email: yuviofficial4491@gmail.com	GET request returns the same user details that were submitted

Want me to convert this into an actual runnable script (Postman collection, or Playwright/REST-assured code) using this same test data?

Observation:

1. **With ICEPOT framework – Output is defined and user will be able to retrieve what they require at the given point**

2. **Output is clear and ready to use, using weak prompt makes LLM to assume and generate exhaustive output at one go**
3. **Applying framework reduces the interaction rate and that way tokens could be saved**